

CONFIDENTIAL — INTERNAL

DRAFT FOR REVIEW

Security Review — July 2026

Internal Adversarial Audit

Evidence discipline. Every finding below cites the test, migration, guard script, or source file that proves the remediation. Where something is *not* directly verifiable from the development environment (most importantly the production runtime and the production database), it is labelled **[INFERRED]** or **[UNVERIFIED]** and repeated in *Known limitations and honest gaps*. “No evidence of X” is never written up as “X did not happen.”

1. Scope and method

Method. Internal white-box adversarial audit by the engineering agent with full source access, run against the **development** environment and the committed migration history. It is a self-review, not a third-party penetration test.

In scope:

- **All HTTP routes** in `artifacts/api-server/src/routes/` (267 id-bearing routes across 64 route files, per the route-ownership guard scan output).
- **Object/recording storage** access paths (`artifacts/api-server/src/routes/storage.ts`).
- **In-memory state** surfaces (agent orchestrator run/event state read by the agent dashboard routes).
- **Privacy / cross-tenant correlation** of the shared “platform” candidate pool (the job-seeker database shared across employer tenants).

Out of scope / not reachable from dev:

- The **production runtime** and **production database** (no dev-side network path to prod; production deployment logs were empty at review time). All production statements are **[INFERRED]** or **[UNVERIFIED]** — see §4–§5 and the companion *Production Verification Checklist*.
- Third-party provider internals (PDL, SERP, Azure, HeyGen).

How claims were verified. By running the automated test suite (`artifacts/api-server/src/**/*.test.ts`, 36 test files) and the build-gate CI guards (`artifacts/api-server/scripts/check-*.mjs`), and by reading the cited source and migration files.

2. Findings and remediation

Severity key: **Critical** = cross-tenant / job-seeker data exposure; **High** = missing authorization on a data surface; **Medium** = integrity / governance / audit weakness.

#	Sev	Finding	Fix	Proof (test / migration / file)	Date
6	Critical	Job-seeker privacy leak (flagship). Employer-facing reads of the shared platform candidate pool returned platform-pool job-seekers without the privacy seal , so candidates who had set <i>hide-from-employer</i> , <i>discovery-paused</i> , <i>blocked-company</i> , or <i>match-only</i> — plus <i>do-not-contact</i> / <i>GDPR-erased</i> rows — could be surfaced to employers who should never see them. Root cause: the mental model was “two boxes” (tenant rows vs platform rows) which was incorrect ; several employer reads treated a candidate-database <i>licence</i> as if it were a <i>visibility</i> grant.	Introduced ONE canonical seal — <code>applyCandidateHardExclusions()</code> (drops erased/DNC/pending) plus <code>applyCandidatePrivacyFilter(rows, viewerTenantId)</code> (per-employer hide/pause/block/match-only) — and routed every employer-facing platform read through it, including the two recommendation-push engines. A by-id fetch of a sealed-out candidate returns 404 (existence not disclosed).	<code>artifacts/api-server/src/routes/candidates.ts</code> (both helpers); <code>artifacts/api-server/src/lib/platform-recommendation-engine.ts</code> (<code>rec-push</code> seal in <code>shared evaluateJobAgainstCandidates</code>); <code>tests candidate-database-privacy-seal.test.ts</code> , <code>candidate-privacy-seal-combinatorial.test.ts</code> , <code>candidate-privacy-seal-recommendation.test.ts</code> ; <code>guard scripts/check-platform-pool-read.mjs</code> ; <code>memory .agents/memory/platform-pool-privacy-seal.md</code>	2026-07-06 → 2026-07-07

#	Sev	Finding	Fix	Proof (test / migration / file)	Date
1	High	Tier 1 — unauthenticated / unscoped routes. Id-bearing API routes without a caller-resolution or ownership marker could read/act across tenants.	Every id-bearing route must carry an access-control marker or a named exemption; enforced at build time. The DB layer also fails closed — a route that reaches the database without a resolved tenant context is refused rather than served with admin scope.	Guard scripts/check-route-ownership.mjs (build output: “scanned 64 route files, 267 id-bearing routes ... all carry an access-control marker or named exemption”); pattern doc artifacts/api-server/docs/SECURITY_PATTERNS.md ; test agent-parse-tenant-gate.test.ts	2026-07-05
2	High	Tier 2 — recruiter-ownership ceiling. A plain recruiter could reach candidates/requisitions not assigned to them (tenant scope alone was insufficient).	Candidate/req routes gate through recruiterCanAccessCandidate() (defined in routes/candidates.ts) backed by getRecruiterAssignedJobIds() (lib/tenantUtils.ts), fail-closed, with a hard 401/404 posture.	artifacts/api-server/src/routes/candidates.ts (recruiterCanAccessCandidate); artifacts/api-server/src/lib/tenantUtils.ts (getRecruiterAssignedJobIds); tests recruiter-ownership-sweep.test.ts , recruiter-admin-permissions.test.ts , recruiter-admin-extended-permissions.test.ts , recruiter-admin-intel-analytics-scoping.test.ts , recruiter-admins.test.ts	2026-07-05
3a	High	Tier 3 — learning analytics (learning.ts) non-RLS reads. The learning/analytics GET endpoints read candidate_job_intelligence (a NON-RLS table); without an explicit predicate the DB proxy would not tenant-filter them.	Added caller + role gate and an explicit tenant predicate to each learning read.	artifacts/api-server/src/routes/learning.ts ; test learning-read-scoping.test.ts ; memory .agents/memory/learning-analytics-nonrls-scoping.md	2026-07-05
3b	High	Tier 3 — recording /recording/part ownership. The S3 key was derived from a caller-supplied sessionId ; a 401-only check let an authenticated caller touch another session’s recording parts.	Ownership is checked via shared isCallerAuthorizedForSession() before any S3 I/O (routes/storage.ts); unauthorized + missing both return 404.	artifacts/api-server/src/routes/storage.ts ; test storage-recording-part-scoping.test.ts ; memory .agents/memory/recording-part-ownership-gate.md	2026-07-05
4	Medium	Stage-write / audit chokepoint. Pipeline-stage transitions written ad hoc could bypass governance (fairness gate, audit, adverse-action controls).	All pipeline-stage transitions route through changeCandidateStage() ; a build guard forbids ungoverned adverse stage writes and off-chokepoint transitions.	artifacts/api-server/src/lib/change-candidate-stage.ts ; guards scripts/check-stage-choke-point.mjs (“all pipeline-stage transitions route through changeCandidateStage()”) and scripts/check-no-adverse-writes.mjs (“no ungoverned adverse stage writes found”)	2026-07-06 (chokepoint guard; adverse-write guard 2026-05-17)

Note on “Row #6”. The flagship privacy finding was tracked during the incident as the sixth row of the read-surface enumeration; it is listed first here because it is the highest severity. The other enumerated read surfaces were confirmed either already-sealed, aggregate-only, or self-directed candidate messaging (see §3, permanent control *platform-pool-read*).

3. Permanent controls

These are standing, build-enforced or schema-enforced controls — not one-off fixes.

CI guards (run in the api-server build gate — build fails if any fails). Wired in artifacts/api-server/package.json build script:

1. **check:route-ownership** (`scripts/check-route-ownership.mjs`) — every id-bearing route must carry an access-control marker or a named exemption. Tracks a **KNOWN GAP: baseline allowlist** (10 route entries at review time: `agents.ts` `pipeline-config/interview-direction/pipeline-status/pipeline-stop`, `communication.ts` `events/ghosting-risks`, `learning.ts` `source-quality`) that is **re-printed loudly on every build** so it stays visible until closed. These are a *route-ownership* concern, distinct from the privacy leak.
2. **Stage-write governance** — `check:stage-choke-point` (`scripts/check-stage-choke-point.mjs`) forces all stage transitions through `changeCandidateStage()`, and `check:no-adverse-writes` (`scripts/check-no-adverse-writes.mjs`) forbids ungoverned adverse stage writes.
3. **check:platform-pool-read** (`scripts/check-platform-pool-read.mjs`) — every read whose predicate INCLUDES `pool='platform'` must be **sealed**, carry a **named exemption** (`PLATFORM_READ_EXEMPTION` in `lib/platform-pool-read.ts`), or be a **reviewed allowlist** entry. Steady state at review time: **9 reads = 3 sealed + 4 named-exempt + 2 allowlisted (VERIFIED-CONTROLLED)**, exit 0.
 - Named exemptions verified as genuinely different risk classes: `SELF_DIRECTED_CANDIDATE_MESSAGING` (a candidate emailing *themselves*, in `candidate-reengagement-scheduler.ts` and `weekly-digest-scheduler.ts` — both exclude erased rows at the query via `isNull(dataErasedAt)` and suppress do-not-contact per-row) and `AGGREGATE_ANALYTICS_COUNT` (two aggregate-count reads in `analytics.ts` that never return per-candidate PII to an employer and are gated on candidate-database access).
 - **By-id blind spot (documented limitation)**. This guard only detects reads that literally filter `pool='platform'`. An employer-facing fetch **by candidate id** does not, so the guard cannot see it. Those routes were hand-audited this review — detail-by-id fails closed (404 for all platform rows because `getDataScopeTenantIds()` never returns "platform"); career-recording and career-profile apply the access gate **plus** the full seal (404 on a filtered row); the by-id message route is 403 cross-tenant. This blind spot is a **standing limitation** carried as a **follow-up ticket** (see §4, gap #4), because a *future* by-id read could reintroduce a leak without tripping the guard.

(Two further guards run in the same gate — `check:no-console` and `check:deletion-cascade-drift` — for hygiene/data-lifecycle, not tenant authorization.)

Row-Level Security (RLS). Migration `lib/db/drizzle/0000_rls_pilot.sql` enables ROW LEVEL SECURITY and FORCE ROW LEVEL SECURITY on `candidates`, `applications`, and `interview_sessions`, and creates a `tenant_isolation` policy on each (`USING/WITH CHECK: current_setting('app.is_platform_admin', true) = 'true' OR tenant_id = current_setting('app.current_tenant_id', true)`). RLS is extended to further tables in `0001_rls_extension.sql` and carve-outs are documented in `0004_rls_carveouts_bespoke.sql`. **This describes the migration intent, not the running state**. Read read-only against the dev DB, only 4 tables actually enforce RLS (live policy) today; the tables above carry a `FORCE` flag with the policy stripped (the `drizzle-kit` push residue), and a whole class of candidate-data tables was never in any RLS migration at all. **The production posture is [UNVERIFIED]** — see the Class A vs Class B breakdown in §5.

Security patterns doc. `artifacts/api-server/docs/SECURITY_PATTERNS.md` records the canonical authorization patterns the guards enforce.

Attributed run/event audit trail (with retention). Agent/pipeline runs and their events are persisted (`lib/db/drizzle/0039_agent_runs.sql`, `0043_pipeline_run_events.sql`), and the acting user is recorded via the `triggered_by_user_id` column added by `0042_pipeline_run_triggered_by_user.sql` (on `pipeline_runs`) — the durable “who”. Non-milestone pipeline events are pruned after **90 days** by the pipeline-events retention scheduler.

HTTP access logging (with retention). `pino-http` access logs are persisted to `http_access_logs` (`lib/db/drizzle/0045_http_access_logs.sql`). Captured: method, **registered route pattern (NULL for unmatched — never derived from the raw URL/querystring)**, status, response time, ip, resolved user/tenant id, request id. **Deliberately NOT captured:** request bodies, tokens, auth headers, query strings, raw URL/path. Pruned after **30 days**. This is a platform-operations table (admin-pool writes, not RLS-protected, no tenant-facing route reads it raw).

4. Known limitations and honest gaps

Stated plainly, not softened. These are the things a reviewer should not assume away.

1. **Production safety is [INFERRED], not confirmed**. All remediation above was verified in **development** (tests + guards + source). There was no dev-side path to the production runtime, and production deployment logs were empty at review time. Whether the sealed code path is actually the one serving prod traffic is **not confirmed here** — it must be checked with prod access (see the companion *Production Verification Checklist*).
2. **Access-log coverage has a start boundary; earlier exposure is unknowable**. Durable, queryable HTTP access logging (the `http_access_logs` table) was introduced by migration `0045_http_access_logs.sql`, committed **2026-07-06**

21:06 UTC. The date it began recording **in production** is itself **[UNVERIFIED]** (depends on the prod deploy of that migration). Regardless, **any request before durable logging existed is not retrospectively queryable** — we cannot prove or disprove pre-logging access to the previously-leaky endpoints. We do **not** claim “no prior access.”

- 3. The privacy tests prove seal *composition*, not live end-to-end AI.** The recommendation-push test (`candidate-privacy-seal-recommendation.test.ts`) proves the exact seal composition/order and per-employer tenant argument that the engine applies, and the guard proves the wiring. It does **not** execute the full recommendation pipeline with live model scoring: `mock.module` is unavailable under this tsx/Node runtime, and a real-engine test would make non-deterministic, billable AI calls over the whole pool. This is a deliberate containment-regression strategy, not full E2E proof.
- 4. The `check:platform-pool-read` guard has a by-id blind spot.** It only sees reads that filter `pool='platform'` . By-id employer-facing candidate reads were hand-audited this review and are safe today, but a future by-id read could reintroduce a leak without failing the build. Carried as a follow-up ticket; requires either periodic manual audit or an extended guard.
- 5. Per-provider external spend (item 4c) is not itemized.** External sourcing provider spend (PDL, SERP, etc.) is not broken out per provider in a verifiable ledger here; sourcing-provider spend is a **known gap** (see `.agents/memory/pdl-tiered-location-relaxation.md`). No spend claim is made in this review.

(The production RLS posture on `candidates` was formerly listed here as a sixth gap; it has been promoted to its own callout — see §5 below — because it may be the single highest open risk, not a routine caveat.)

5. Open critical question — production RLS posture (Class A vs Class B)

How is tenant isolation actually enforced — at the database (RLS) or by application code alone? The answer differs by table, and for a whole class of candidate-data tables the answer is “app-code only, in every environment, by design.”

Verified read-only against the **development** DB (`pg_class` / `pg_policies`): of ~104 public tables, **only 4** have working RLS (`relrowsecurity = t` + a live policy) — `agent_runs`, `agent_run_events`, `job_recruiters`, `pipeline_run_events` — the tables whose raw `.sql` migrations inline their own `ENABLE + FORCE + CREATE POLICY` . **Every other tenant/candidate-data table has `relrowsecurity = f` and zero policies in dev**, so in development RLS enforces nothing for essentially all candidate data and **app code is the sole tenant seal**. (The `db` proxy still sets the `app.current_tenant_id` / `app.is_platform_admin` GUCs, but with the policies absent nothing reads them; `dbAdmin` is `BYPASSRLS` by design.)

The `relforcerowsecurity` flag discriminates two classes:

- Class A (`rls_enabled = f`, `rls_forced = t`) — RLS *intended*.** Tables an RLS migration meant to protect (`0000_rls_pilot.sql`: `candidates`, `applications`, `interview_sessions`; `0001_rls_extension.sql`: `jobs`, `sourced_candidates`, `talent_matches`, `resume_screens`, `outreach_`, `recruiter_inbox_items`, `verification_records`, `interview_plans/schedules`, `ghosting_`, `nurture_pool`, `pipeline_runs`, `job_pipelines`, `prep_`, `notifications`, `communication_events`, `candidate_rejections`, `candidate_import_`, `credit_usage_events`, `tenant_decision_policies`, `billing_*`; + `0004 bespoke`). The lone `FORCE` flag with **zero policies is residue** — the policy was stripped. A **correctly-migrated production would have RLS here**, so for Class A the dev gap is *expected* to be dev-only — but that is **[UNVERIFIED]** from development.
- Class B (`rls_enabled = f`, `rls_forced = f`) — RLS *never existed*.** Candidate-data tables that appear in **no** RLS migration, so no policy exists even in the intended design → **application code is the sole tenant seal in every environment, production included, by design**. Includes `candidate_job_intelligence`, `hiring_manager_shares`, `interview_summaries`, `candidate_career_profiles`, `candidate_demographics`, `candidate_ai_consent`, `candidate_embeddings`, `candidate_skill_scores`, `candidate_events/outcomes`, `outreach_replies` / `conversation_drafts` / `sequence_steps` / `step_messages`, `connection_`, `decision_events`, `ai_decision_log`, `stt_transcribe_events`, `workorder_ai_` / `tenant_ai_*`.

Why this matters for this incident: the two cross-tenant findings in this review — `candidate_job_intelligence` (by-candidate reads) and `hiring_manager_shares` — are **Class B**. There is **no DB backstop for them in production**; they leaked in prod too, not merely in a dev test harness, and the app-layer fixes in §2 (plus the new `check:classb-read` build-gate guard) are the *only* thing sealing them anywhere. The pipeline-board `sourced_candidates` exposure is **Class A** → dev-only if prod RLS is intact. This reframes the earlier “one open question about `candidates` in prod” into a **platform-wide class distinction**.

Why RLS is missing (systematic, not ad hoc): the (`f`, `t`) `FORCE`-residue + zero-policies pattern is the `drizzle-kit push` fingerprint — push reconciles to the drizzle schema, which declares the `force/enable` table config but **not** the `CREATE POLICY` statements (those live only in raw `.sql`), so push drops policies. Only later raw-`.sql` tables kept their RLS.

What is still open (and only prod can answer): for **Class A** tables, whether production actually has the policies or shares the dev `(f, t)` residue — where `relrowsecurity = f` means RLS is **not actually enabled** (the lone `FORCE` flag is inert), so rows are unfiltered and isolation is **app-code-only** — is **[UNVERIFIED]** — prod is an external DB, unreachable from the workspace, so it must be checked at/after Publish. Run **Item 3** of the companion *Production Verification Checklist* (now generalized from `candidates` to the Class-A set):

- **policy present** → DB-enforced for that table; or
- **FORCE RLS with zero policies** (or RLS off) → isolation is app-code-only; escalate and apply the `.sql` policy migration (never `drizzle-kit push`).

For **Class B** tables there is nothing to verify in prod — by design no DB policy exists anywhere; the app-layer seal (and its CI guards) is the whole control, so any future regression there re-opens exposure with no DB backstop.

6. Bottom line

In **development**, the flagship job-seeker privacy leak and the Tier 1–3 authorization findings are remediated, each backed by a named test/guard/file, and the fixes are protected going forward by build-gate CI guards, a DB-migration RLS baseline, and attributed, retention-bounded audit + access logs. The remaining honest gaps are almost entirely about **production confirmation** — is the sealed code actually live, and (the open critical question in §5) are the **Class-A** tables actually policy-backed in prod (Class-B tables are app-code-only by design, with no DB backstop to verify) — which cannot be answered from dev and are handed off in the companion *Production Verification Checklist*.

Author note (for reviewers deciding on distribution). This document was produced by Engineering as an internal self-review, generated with Replit Agent on the `main` branch. That provenance is disclosed deliberately for internal use. **If this document is shared externally** (customer, auditor, questionnaire response), simplify the author line to **“Engineering”** and drop the tooling disclosure — an external reader should see an organizational author, not the internal authoring method. The recommendation is: **keep the disclosure for internal circulation; switch to “Engineering” before any external release.**